

Laboratório de Concorrência · Aula 10

Arthur Lavagnolli Molina · RA 24031565-2 · Turma ESOF 6S "A"

Gerado em 10/10/2026, 01:28:55 · tempo registrado no cronômetro: 0:00

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Sincronização de cooperação é quando um processo depende do outro para sua continuidade, ou seja, o processo 1 espera o processo 2 finalizar para continuar, e vice-versa.

Sincronização de competição é quando 2 ou mais processos precisam acessar o mesmo recurso ao mesmo tempo, gerando a condição de corrida, pois usará o recurso aquele que "chegar primeiro".

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

2.000.000

1ª execução (n = 1.000.000)

1028712

2ª execução (n = 1.000.000)

1430049

3ª execução (n = 1.000.000)

1007553

Saída do programa

```
Saída obtida com n = 1.000.000
esperado: 2000000
obtido: 1028712
```

```
esperado: 2000000
obtido: 1430049
```

```
esperado: 2000000
obtido: 1007553
```

```
saída obtida com n = 1000
esperado: 2000
obtido: 2000
```

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não foram conforme esperado, nas três execuções foram diferentes e mudaram em cada execução devido o escalonamento das threads que fizeram com que a variável mudasse o valor quando o escalonador parou a primeira thread para iniciar a segunda

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

um exemplo do que pode acontecer para resultar nesse valor, t1 é iniciado e lê a variável contador com valor 0 por exemplo, e então o escalonador muda para a execução de t2, que lê a variável contador com valor 0 também, em seguida volta para t1 e soma e grava, deixando a variável com valor 1, após isso, executa t2 que irá somar e gravar o que leu que foi 0, incrementando e deixando o valor como 1 também. Neste caso, as duas threads incrementaram porém o valor continuou como 1 no contador

1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Devido ao baixo volume de iteração de incremento do loop, ao iniciar a t2, a t1 já terminou de incrementar, fazendo com que a soma finalize de forma correta. Isso é perigoso pois é um resultado subjetivo, condicionado a condição do escalonador, podendo surtir efeitos inesperados no sistema. Um caso como esse pode funcionar normal nos teste e na produção gerar algum erro, devido a incerteza que esse tipo de desenvolvimento sem o controle adequado gera.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB → 8
ABABAB → 6
AABBAB → 6
BBBAAA → 7
BBAABA → 4
ABAABB → 6
AABBBA → 4

Saída do programa

```
AAABBB → 8
BBBAAA → 7
ABABAB → 6
Ordens possíveis: 20
TOTAL = 4: 9 ordens, por exemplo AABBBB
TOTAL = 6: 9 ordens, por exemplo AABABB
TOTAL = 7: 1 ordens, por exemplo BBBAAA
TOTAL = 8: 1 ordens, por exemplo AAABBB
```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

são possível 20 possíveis combinações com 4 resultados possíveis. A ordem do resultado correto é AAABBB

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

AABBBA

total + 1 lê = 3

total + 1 calcula = 4

total * 2 lê = 3

total * 2 calcula = 6

total * 2 grava = 6

total + 1 grava = 4

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Acho que a operação atômica será mais rápida e nenhuma será incorreta.

Saída do programa

Saída da primeira execução:

```
esperado:          20000000
sem sincronizacao: 1008083 em 2 ms
synchronized:      20000000 em 110 ms
AtomicInteger:      20000000 em 62 ms
```

Saída da segunda execução:

```
esperado:          20000000
sem sincronizacao: 1017793 em 2 ms
synchronized:      20000000 em 179 ms
AtomicInteger:      20000000 em 27 ms
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

Ambas, a versão correta costuma ser mais lenta pois precisa fazer o controle de sincronização

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Nesse caso não seria possível, pois para esse caso precisaria de sincronização de cooperação, pois envolvem duas contas distintas e a falta dessa sincronização nas contas pode gerar dados inconsistentes. Seria preciso bloquear as duas contas ao mesmo tempo para impedir algumas movimentação inesperada.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Não, pois uma fica aguardando a outra finalizar e liberar o recurso

Saídas (antes e depois da correção)

Saída da primeira execução:

```
IMPASSE: 2 threads presas
```

Saída da segunda execução após correção:

```
A terminou
B terminou
sem impasse
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

Ficaram presas devido deadlock, que ocorre quando duas ou mais threads esperam o recursos uma da outra e acabam entrando em um impasse.

A tinha x e esperava o recurso y

b tinha y e esperava o recurso x

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

O impasse só ocorre quando há um espera circular, ou seja, se todos os processos pedem os recursos em uma ordem linear, não é possível formar um impasse.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

3 trabalhadores 200ms

1 trabalhador 600ms

6 trabalhadores 100ms

Tempo com 1 trabalhador

600ms

Tempo com 3 trabalhadores

200ms

Tempo com 6 trabalhadores

100ms

Saída do programa

saída 1 trabalhadores:

trabalhador 1 fez a tarefa 1

trabalhador 1 fez a tarefa 2

trabalhador 1 fez a tarefa 3

trabalhador 1 fez a tarefa 4

trabalhador 1 fez a tarefa 5

trabalhador 1 fez a tarefa 6

1 trabalhadores, 6 tarefas de 100 ms: levou 600ms

saída 3 trabalhadores:

trabalhador 3 fez a tarefa 1

trabalhador 1 fez a tarefa 2

trabalhador 2 fez a tarefa 3

trabalhador 3 fez a tarefa 4

trabalhador 2 fez a tarefa 6

trabalhador 1 fez a tarefa 5

3 trabalhadores, 6 tarefas de 100 ms: levou 200ms

saída 6 trabalhadores:

trabalhador 2 fez a tarefa 3

trabalhador 6 fez a tarefa 1

trabalhador 1 fez a tarefa 2

trabalhador 3 fez a tarefa 4

trabalhador 4 fez a tarefa 5

trabalhador 5 fez a tarefa 6

6 trabalhadores, 6 tarefas de 100 ms: levou 100ms

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Devido o acesso na memória não ser compartilhado por threads, aqui em go ele trabalha com trocam de mensagens para as operações, impedindo o impasse.

Saídas (antes e depois da troca)

```
antes da troca:
fatal error: all goroutines are asleep - deadlock!
```

```
goroutine 1 [chan send]:
main.main()
    /box/main.go:9 +0x38
exit status 2
```

```
depois da troca:
1
```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

Acontece que em `ch <- 1`, o valor para o canal, porém o mesmo está sem espaço para o canal, ocasionando um deadlock pois o valor é enviado e aguardado o recebimento do valor, que acaba não acontecendo devido a falta de espaço para o canal.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

O programa não vai permitir a compilação

Saídas (o erro e o resultado com move)

```
o erro:
error[E0373]: closure may outlive the current function, but it borrows `contador`, which is
owned by the current function
  → HelloWorld.rs:7:27
   |
7 |     let t = thread::spawn(|| {
   |                               ^^ may outlive borrowed value `contador`
8 |         contador += 1;           // a thread altera a variável de main
   |         ----- `contador` is borrowed here
```

```
resultado com move:
warning: value captured by `contador` is never read
  → HelloWorld.rs:8:9
   |
8 |         contador += 1;           // a thread altera a variável de main
   |         ^^^^^^^^^
   |
= help: did you mean to capture by reference instead?
= note: `#[warn(unused_assignments)]` (part of `#[warn(unused)]`) on by default
```

Saída do programa

```
contador = 400000
```

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

devido a falta de acesso a memória compartilhada, ele faz esse tipo de operação, evitando o deadlock

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Java: em execução

Rust: na compilação

A verificação na execução pode acarretar em um erro passado despercebido pelo time de teste, já a verificação na compilação evita erros, embutindo segurança ao sistema, para esse cenário o custo é a exigência e a curva de aprendizado, pois precisará lidar com regras definidas do compilador

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A,B,C e D

Saída do programa

```
A
D
C
B
E: timer de 10 ms rodou depois de 300 ms
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

```
A
D
C
B
```

A intrução para impressão de D era síncrona, executando imediatamente na pilha principal. Já a intrução para impressão de C era de prioridade mais elevada pois é uma "Promise" e não um temporizador como a intrução que imprimia B

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

Porque o laço de repetição síncrono travou a única thread do javascript por 300ms, impedindo que o temporizador que expirou, fosse retirado da fila e executado.

Sobre o servidor Node.js, isso demonstra que cálculos pesados ou código bloqueando na thread, pode congelar por completo o sistema, atrasando ou bloqueando o uso de todos os usuários do sistema simultaneamente.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo.

Qual problema desta atividade pode acontecer? Como você o evitaria?

Ocorre uma condição de corrida, resultando em compra duplicada porque ambas visualizaram o lugar disponível antes de o sistema atualizar.

Para evitar esse tipo de situação deve-se usar mecanismo de situação mútua, como vimos nesta aula, a fim de garantir que apenas uma requisição consiga efetuar a reserva daquele único lugar.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

As falha que uma programação sem considerar conceitos de condições de corrida pode ocasionar.